Workflow Agentic Automation

Complete Documentation: 9 Sections (14-22)

Upstream Prevention · Detailed Diagnostics · Automation Fixing Best Practices

📚 Documentation Sections

1. **14.** Executability Principle
2. **15.** Test Failure Reporting
3. **16.** Assertion Design
4. **17.** Traceability Record
5. **18.** End-to-End Flow (PRD → Automation)
6. **19.** PR Review Prerequisites
7. **20.** Fix PR from Feedback
8. **21.** Environment Confirmation (Dev → STG)
9. **22.** Automation Fixing Best Practices ← NEW!

14 — Executability principle: No assumptions

Goal: EVERYTHING must be EXECUTABLE in dev and stg. No aspirational behavior. Every layer verified in real environment before claiming automation.

Core Rule

> NOT: "Assume the code will work, test later"
> BUT: "Verify everything works in REAL environment, then write code"

What Must Be Executable

Layer	Must Be Executable	Verify By
SPEC	test.step, expect() runs in real env	Run test in dev/stg
USECASE	Endpoint call works, response valid	Grounding: endpoint exists, responds
Layer 3	Selector found, element exists	Grounding: live inspection of app
FACTORY	Generated data insertable	Test: data passes validation, inserts
PRE	Setup endpoint works, DB mutates	Grounding: endpoint exists, verify response
Environment	Dev and stg have SAME structure	Verify: endpoint URL, schema, data

---

Per-Layer Executability Rules

Layer 1: SPEC

✅ Executable:

```
test('@covers DF-M1-POS-001', async () => {
  const data = factory.generateEmployee()
  const result = await usecase.execute(data)
  expect(result.status).toBe('DRAFT')  // Actual value, not assumption
})
```

❌ NOT executable (assumption):

```
test('Create employee', async () => {
  // Assume usecase.execute works
  const result = await usecase.execute({ name: 'John' })
  expect(result).toBeDefined()  // Assume something will be defined
})
```

Checklist: - [ ] Usecase call = real function (not mock) - [ ] Expected value = from TC (not from code) - [ ] Assertion = meaningful, can fail - [ ] No assumption of success

---

Layer 2: USECASE

✅ Executable:

```
export class CreateEmployeeUsecase {
  async execute(data: EmployeeInput): Promise<UsecaseResult> {
    // Call REAL endpoint
    const res = await http.post('/v1/employees', data)

    // Validate response (not assume success)
    if (!res.ok()) {
      throw new Error(`API error: ${res.status()}`)
    }

    return { success: true, data: res.json() }
  }
}
```

❌ NOT executable (assumption):

```
export class CreateEmployeeUsecase {
  async execute(data) {
    // Assume endpoint exists (not verified)
    // Assume response has data field
    const res = await http.post('/v1/employees', data)
    return res  // Assume caller will handle error
  }
}
```

Checklist: - [ ] HTTP call = to real endpoint (not mock) - [ ] Response validated = check status, structure - [ ] Error handling = explicit throw (not silent) - [ ] No assumption of success

---

Layer 3: PAGE/SCREEN/COMPONENT

✅ Executable:

```
// src: kerjaan-client-portal/src/pages/CreateEmployee.tsx:42
async fillName(name: string) {
  // Selector verified in grounding (live inspection)
  await this.page.fill('[data-testid="employee-name"]', name)

  // Wait for actual network (not fixed sleep)
  await this.page.waitForLoadState('networkidle')
}
```

❌ NOT executable (assumption):

```
async fillName(name: string) {
  // Assume selector exists (not verified)
  await this.page.fill('[data-testid="employee-name"]', name)

  // Assume 2 seconds is enough (hardcoded wait)
  await this.page.waitForTimeout(2000)
}
```

Checklist: - [ ] Selector = grounded (live inspected app) - [ ] Selector = commented with // src: - [ ] Wait = dynamic (waitForLoadState, not waitForTimeout) - [ ] No assumption of element existence

---

FACTORY

✅ Executable:

```
export function generateEmployee(overrides = {}) {
  // Generate varied, valid data
  return {
    name: faker.person.fullName(),        // Unique per run
    email: faker.internet.email(),        // Valid format
    salary: faker.number.int({ min: 3000000, max: 10000000 }),
    ...overrides
  }
}
```

❌ NOT executable (assumption):

```
export function generateEmployee() {
  // Hardcoded same data every run
  return {
    name: 'John Doe',
    email: 'john@test.co',
    salary: 5000000  // Assume this is valid
  }
}
```

Checklist: - [ ] Data = varied (not hardcoded) - [ ] Data = schema-valid (email format, constraints) - [ ] No API call in FACTORY - [ ] No assumption of data uniqueness

---

PRE (Precondition)

✅ Executable:

```
export async function preconditionManagerUser() {
  // PRE endpoint verified in grounding
  const response = await request.post('/v1/admin/seed/user', {
    role: 'manager',
    email: `mgr-${Date.now()}@test.co`  // Unique per run
  })

  // Assert PRE actually worked
  expect(response.status()).toBe(201)
  expect(response.json().id).toBeTruthy()

  return response.json()
}
```

❌ NOT executable (assumption):

```
export async function preconditionManagerUser() {
  // Assume endpoint exists (not verified)
  // Assume it always succeeds
  const response = await request.post('/v1/admin/seed/user', {...})
  return response  // No assertion, assume success
}
```

Checklist: - [ ] PRE endpoint = verified in grounding - [ ] PRE response = validated (status, data) - [ ] PRE data = unique (not reused) - [ ] PRE assertion = explicit (expect status, data)

Environment Parity

Rule: All layers tested in SAME environment (dev/dev or stg/stg).

```
❌ WRONG:
- PRE tested in dev ✓
- Scenario tested in stg ✗
- Endpoint different: dev localhost, stg api.staging.co
- Test fail, unclear why

✅ RIGHT:
- PRE tested in dev
- Scenario tested in dev
- OR both in stg
- Same endpoint, same schema, same behavior
```

Verify:

```
# Check endpoint URL
echo $API_ENDPOINT  # Should be dev or stg, not mixed

# Check environment config
grep -r "localhost\|staging" code/  # Should use ENV, not hardcoded

# Test in target env only
ENVIRONMENT=dev npm test
ENVIRONMENT=stg npm test
# Not mixed!
```

Checklist: Executability per Phase

Phase 1: Grounding

```
[ ] Selector: Live inspection of app, element FOUND
[ ] Endpoint: Search codebase, URL EXISTS
[ ] Expected value: From TC or app response, not memory
[ ] Error scenario: Can this be triggered in real env?
[ ] PRE: Endpoint reachable, response structure known
```

Phase 2: Design

```
[ ] Assertion: Can fail? Meaningful? From TC?
[ ] Error handling: How to detect failure per step?
[ ] PRE sequence: Setup will work in target env?
[ ] Data flow: Data from PRE → scenario → assertion clear?
```

Phase 4: Implement

```
[ ] SPEC: Calls usecase only (not page/API direct)
[ ] USECASE: Calls real endpoint, validates response
[ ] Layer 3: Selector grounded (// src:), no hardcoded
[ ] FACTORY: Data varied, valid schema, not API call
[ ] PRE: Endpoint real, response validated, unique data
```

Phase 5: Verify

```
[ ] PRE: Does setup endpoint respond? DB mutates?
[ ] Test: Does run in target environment? PASSED?
[ ] Selector: Does live inspection confirm element found?
[ ] Response: Does actual response match expected?
[ ] Error: If fail, is error clear (not assumption)?
```

Anti-Patterns: What NOT to Do

> ❌ "The endpoint probably works, let's write test"
> ✅ "Verified endpoint works, now write test"
>
> ❌ "Selector should be there, let me hardcode"
> ✅ "Grounded selector with live inspection, now hardcode"
>
> ❌ "Expected value looks right from code"
> ✅ "Expected value from TC Then clause"
>
> ❌ "Test passed locally, must be correct"
> ✅ "Test passed locally AND in stg, then claim automation"
>
> ❌ "PRE setup probably works"
> ✅ "PRE endpoint responds + DB mutation verified"
>
> ❌ "Generate data same every run"
> ✅ "Generate data varied, unique per run"

Summary: The Principle

> EXECUTABILITY PRINCIPLE:
>
> Every assertion, endpoint, selector, value, and precondition
> must be VERIFIED IN REAL ENVIRONMENT,
> not assumed or hoped.
>
> Code is EXECUTED, not THEORETICAL.
>
> If something cannot be executed in real environment,
> it's not part of the automation — it's blocked.

Reference

- Grounding: `docs/shared/03-codebase-grounding.md`
- Layer architecture: `docs/shared/04-three-layer-architecture.md`
- Test design: `docs/shared/05-test-design-rules.md`
- Assertion design: `docs/shared/16-assertion-design.md`
- Failure reporting: `docs/shared/15-test-failure-reporting.md`

---

15 — Test failure reporting: Detailed error debugging

Goal: Ketika test gagal, error harus dilaporkan dengan detail lengkap dan actionable, bukan generic "test failed". Setiap failure harus traceable ke TC, layer, dan root cause yang jelas.

Rule Utama

1. **Gagal = Detail.** Error message, stack trace, screenshot, video — semuanya harus attached.
2. **Layer identification.** Harus jelas: PRE fail? FACTORY fail? USECASE fail? Layer 3 fail? SPEC assertion fail?
3. **@covers TC-ID required.** Error harus link ke TC yang sedang dijalankan.
4. **Root cause analysis.** "Kenapa gagal?" harus dijawab, bukan diabaikan.
5. **Decision tree.** Kalau gagal, apa action yang harus diambil? Tetap terstruktur.
6. **No silent failures.** Tidak ada exception ditelan, tidak ada tautology, tidak ada assertion semu.

Failure Report Format (Standard)

```
═══════════════════════════════════════════════════════════════
TEST FAILURE REPORT
═══════════════════════════════════════════════════════════════

@covers: [TC-ID] — [TC Title]
Module: [module-name]
Environment: [dev|stg]
Timestamp: [YYYY-MM-DD HH:mm:ss]
Duration: [XX.XXs]

WHICH LAYER FAILED?
  Layer: [SPEC | USECASE | Layer3-PAGE | Layer3-COMPONENT | FACTORY | PRE]
  Severity: [BLOCKER | MAJOR | MINOR]

TEST STEP SEQUENCE:
  ✅ PRE: setup manager user — PASSED (2.3s)
  ✅ SPEC.step('Navigate'): goto('/form') — PASSED (1.1s)
  ✅ SPEC.step('Fill form'): fillName('John') — PASSED (0.8s)
  ❌ SPEC.step('Submit'): clickSubmit() — FAILED (timeout 30s)
  🔲  SPEC.step('Verify'): checkStatus() — NOT EXECUTED

ERROR MESSAGE:
  Timeout 30000ms exceeded.
  Waiting for locator('button:has-text("Submit")').click()

SELECTOR/ENDPOINT INVOLVED:
  button:has-text("Submit")
  Expected to exist: yes
  Actual found: no
  Waited: 30000ms
  Element visible: no

ROOT CAUSE ANALYSIS:
  Layer 3 PAGE.submitForm() — Selector not found in DOM

  Expected selector (from code):
    domain/web/pm/pages/create-employee.page.ts:42
    await page.click('button:has-text("Submit")')
    // src: kerjaan-client-portal/src/pages/CreateEmployee.tsx:52

  Actual selector (from live inspection):
    button:has-text("Save & Continue")
    (button text berbeda!)
```

```
  Analysis:
    ❌ Grounding phase found: "Submit"
    ✅ Live DOM shows: "Save & Continue"
    → Selector outdated atau app changed

DECISION POINT:
  Option A: Fix selector di PAGE layer
    └─ Command: grep -r "Save & Continue" kerjaan-client-portal/src/
    └─ Verify: re-ground selector dengan app terbaru
    └─ Update: pages/create-employee.page.ts:42

  Option B: Re-check grounding
    └─ App berubah sejak grounding phase?
    └─ Re-ground TC scenario terhadap app terbaru

  Option C: Escalate
    └─ UI text mismatch TC expected vs actual
    └─ Decision: TC salah atau app berubah?

EVIDENCE ATTACHED:
  Screenshot: outputs/evidence/2025-01-15/PM-C-001__create-employee__submit-fail
              (Shows DOM state saat selector tidak ketemu)

  Video: outputs/evidence/2025-01-15/PM-C-001__create-employee__submit-fail.webm
         (Shows full flow sampai timeout)

  Browser console:
    Network: No failed requests before timeout
    Console: No JS errors
    Performance: All steps executed normally until submit button

RUN INFORMATION:
  Command: playwright test domain/web/pm/tests/create-employee.spec.ts -g "PM-C-
  Browser: chromium v130.0.6723.119
  OS: darwin (macOS)
  Node: v20.11.0
  Playwright: 1.48.0

ENVIRONMENT DETAILS:
  URL: https://dev-staffinc.local
  Database: dev postgres (seeded this run)
  Cache: cleared before test
  Network: stable (no timeouts observed elsewhere)
```

```
═══════════════════════════════════════════════════════════════
ACTION REQUIRED: [BLOCKED | NEEDS DECISION | FIX REQUIRED]
Assignee: [automation-engineer | QA-lead | developer]
═══════════════════════════════════════════════════════════════
```

Failure Types & Diagnosis

Type 1: PRE (Precondition) Failed

```
@covers: DF-M1-POS-001
WHICH LAYER FAILED? PRE (Precondition Setup)

ERROR:
  POST /v1/admin/seed/user HTTP 500
  Response body: {
    "error": "Database connection timeout",
    "code": "DB_TIMEOUT"
  }

ROOT CAUSE:
  Precondition endpoint tidak responding di environment

  Endpoint: hcmsvc/internal/admin/handler.go:234
  // src: hcmsvc/internal/admin/handler.go:234

  Expected: HTTP 201, users row inserted
  Actual: HTTP 500, no data inserted

DATABASE STATE:
  Before: users table — 42 rows
  After: users table — 42 rows (no insert)
  Conclusion: PRE mutation did NOT happen

DECISION NEEDED:
  ❌ PRE BLOCKED — Cannot proceed to test scenario
  ❌ TC STATUS REMAINS: planned (NOT automated)

  Action items:
    1. Check database connection in [environment]
    2. Check /v1/admin/seed/user endpoint uptime
    3. Retry PRE after environment fixed
    4. Only then try test scenario

NEVER:
  ❌ Skip PRE and "assume" it worked
  ❌ Use hardcoded user ID yang mungkin sudah exist
  ❌ Mark TC as "automated" kalau PRE tidak setup
```

Type 2: FACTORY Data Invalid

```
@covers: DF-M1-POS-002
WHICH LAYER FAILED? FACTORY (Test Data Generation)

ERROR:
  Data validation failed during INSERT:

  INSERT INTO employees (name, email, role)
  VALUES ('John Doe', 'john@test', 'employee')

  Error: Invalid email format 'john@test'
  Constraint: email LIKE '%@%.%'

FACTORY CODE:
  domain/web/pm/factories/employee-data.factory.ts:28

  export function generateEmployee(overrides = {}) {
    return {
      name: faker.person.fullName(),
      email: faker.internet.email(),  ← Generated invalid: "john@test"
      role: 'employee',
      ...overrides
    }
  }

DATA GENERATED (Attempt #1):
  name: "John Doe" ✓
  email: "john@test" ✗ (invalid format)
  role: "employee" ✓

ROOT CAUSE:
  faker.internet.email() occasionally generates invalid format
  (Unlikely but possible with certain seed values)

DECISION:
  Option A: Retry test
    └─ FACTORY failure di 1 attempt = usually one-off
    └─ Run test again, sama FACTORY generate data lagi

  Option B: Enhance FACTORY validation
    └─ Add validator sebelum return data
    └─ Validate email format sebelum INSERT

  Option C: Use static test data
    └─ Replace faker dengan hardcoded valid data
```

```
    └─ Trade-off: less variety, more reliability

RECOMMENDATION:
  Retry test dulu (one-off failure kemungkinan besar)
  Jika terjadi 2+ kali, enhance FACTORY validator
```

Type 3: USECASE Logic Error

```
@covers: DF-M1-POS-003
WHICH LAYER FAILED? USECASE (Business Logic)

ERROR:
  CreateEmployeeUsecase.execute() returned:
  {
    success: false,
    error: "Validation failed: email already exists"
  }

USECASE CODE:
  domain/web/pm/usecases/create-employee.usecase.ts:15

  async execute(data: EmployeeInput): Promise<UsecaseResult> {
    const existing = await query('SELECT * FROM employees WHERE email=?', data.e
    if (existing.length > 0) {
      return createErrorResult('Validation failed: email already exists')
    }
    const result = await http.post('/v1/employees', data)
    return { success: true, data: result.data }
  }

CALL SEQUENCE:
  ✅ FACTORY.generateEmployee()
     └─ Generated: { name: "John Doe", email: "john@test.co" }

  ❌ USECASE.execute(data)
     └─ Check: email "john@test.co" already exists in DB
     └─ Return: error (did not call HTTP endpoint)

ROOT CAUSE OPTIONS:

  Option A: Database contamination
    └─ DB tidak dibersihkan antara test run
    └─ Email "john@test.co" dari test sebelumnya masih exist
    └─ Solution: Clean DB before each test run

  Option B: FACTORY reuse
    └─ Same FACTORY instance reused across tests
    └─ Solution: Generate fresh data per test

  Option C: USECASE validation wrong
    └─ Validation rule tidak sesuai TC expected behavior
    └─ Solution: Review TC, check if "email exists" check is expected
```

```
DECISION TREE:
  1. Check: TC expected behavior apa?
     └─ TC bilang: "Create dengan email unik" → OK (error is expected)
     └─ TC bilang: "Create langsung dengan email any" → FAIL (validation wrong)

  2. Check: Database state
     └─ SELECT COUNT(*) FROM employees WHERE email='john@test.co'
     └─ If > 0: Database contamination
     └─ If = 0: Something else wrong

  3. Check: FACTORY consistency
     └─ Every call generateEmployee() should return unique email
     └─ If not: FACTORY returns same email

NEXT STEPS:
  If validation expected: Mark test as PASSED (error is correct)
  If validation unexpected: Fix USECASE or DB cleanup
```

Type 4: Layer 3 (PAGE/COMPONENT) Failed

```
@covers: DF-M1-POS-004
WHICH LAYER FAILED? Layer 3 — PAGE (UI Interaction)

ERROR:
  Timeout 30000ms exceeded.
  page.fill('[data-testid="employee-name"]')

  Element: [data-testid="employee-name"]
  Status: NOT FOUND

PAGE CODE:
  domain/web/pm/pages/create-employee.page.ts:42

  async fillName(name: string) {
    await page.fill('[data-testid="employee-name"]', name)
  }
  // src: kerjaan-client-portal/src/pages/CreateEmployee.tsx:52

GROUNDING EVIDENCE:
  Grounding phase found:
    ✅ Selector: [data-testid="employee-name"]
    ✅ Located: kerjaan-client-portal/src/pages/CreateEmployee.tsx:52
    ✅ Element visible in live inspection

  Current app state:
    ❌ Selector: [data-testid="employee-name"] — NOT FOUND
    ✅ Selector: [data-testid="first-name"] — FOUND

  Analysis:
    Grounding found "employee-name"
    Live app has "first-name"
    → Selector outdated OR app changed since grounding

ROOT CAUSE:

  Sub-cause A: Selector in code is wrong
    └─ PAGE layer has typo or old selector
    └─ Fix: Update PAGE layer with correct selector

  Sub-cause B: App UI changed
    └─ data-testid attribute changed in app
    └─ Fix: Re-ground selector against current app version

  Sub-cause C: Grounding was wrong
```

```
    └─ Original grounding found wrong selector
    └─ Fix: Re-ground TC scenario from scratch

HTTP INTERACTION (if applicable):
  Before timeout, network calls:
    ✅ GET /form - 200 OK (page loaded)
    ✅ GET /form/fields - 200 OK (schema loaded)
    ❌ After: No more requests (stuck at fill)

DOM INSPECTION (screenshot):
  Live DOM at failure time:
    <form>
      <input data-testid="first-name" />
      <input data-testid="last-name" />
      <input data-testid="email" />
    </form>

  Expected element: [data-testid="employee-name"]
  Found element: [data-testid="first-name"]

DECISION:
  Option A: Fix PAGE layer selector
    └─ Update: [data-testid="employee-name"]
    └─ To: [data-testid="first-name"]
    └─ Verify: Re-run test

  Option B: Re-ground selector
    └─ Run grounding phase again
    └─ Update grounding notes
    └─ Verify selector actually exists in current app

  Option C: Verify TC matches app
    └─ TC says: "Fill employee name field"
    └─ App shows: "Fill first name field"
    └─ Decision: TC or app wrong?
```

Type 5: SPEC Assertion Failed

```
@covers: DF-M1-POS-005
WHICH LAYER FAILED? SPEC (Assertion)

ASSERTION THAT FAILED:
  expect(result.status).toBe('DRAFT')

  Expected: 'DRAFT'
  Actual: 'PENDING'

SPEC CODE:
  domain/web/pm/tests/create-employee.spec.ts:52

  test('@covers DF-M1-POS-005', async () => {
    const result = await createEmployeeUsecase.execute(data)
    expect(result.status).toBe('DRAFT')  ← ASSERTION FAILED
  })

TC EXPECTED RESULT:
  From staffinc-testcase/testcases/web/pm/DF-M1-POS-005.md:

  "Then: Employees harus dibuat dengan status DRAFT"

ACTUAL BEHAVIOR:
  When POST /v1/employees called:
    Response: { id: 123, name: "John", status: 'PENDING' }

  Database: employees table, id=123, status='PENDING'

MISMATCH ANALYSIS:

  TC says: Status DRAFT after create
  Code does: Status PENDING after create

  Question: Kenapa status PENDING, bukan DRAFT?

  Option A: App workflow changed
    └─ Employees now auto-transition to PENDING on create
    └─ This is a DEFECT (app behavior ≠ TC requirement)
    └─ Action: Report defect, update TC, or fix app

  Option B: TC is outdated
    └─ Requirement changed but TC not updated
    └─ Action: Update TC expected result
    └─ But: Verify with Product/QA first
```

```
  Option C: Assertion is wrong
    └─ Maybe code is correct, assertion misread
    └─ But: Unlikely if assertion directly from TC

DECISION TREE (critical):

  🚫 WRONG ACTION:
     ❌ Change assertion to: expect(result.status).toBe('PENDING')
     ❌ Use loose matcher: expect(result.status).toBeTruthy()
     ❌ Comment out assertion
     ❌ Ignore mismatch and mark as "passed"

     ALL OF ABOVE = forgery, test is not valid

  ✅ RIGHT ACTION:
     1. Verify TC: Open staffinc-testcase/testcases/web/pm/DF-M1-POS-005.md
        └─ Check: Expected Result says DRAFT? Confirms

     2. Check codebase: Why is status PENDING?
        └─ Search: hcmsvc/internal/employee/handler.go
        └─ Find: POST /v1/employees endpoint
        └─ Check: Default status = PENDING?
        └─ Find: Is there workflow auto-transition?

     3. Decide with team:
        A) App is correct: Update TC expected result
           └─ Edit: staffinc-testcase/testcases/web/pm/DF-M1-POS-005.md
           └─ Change: Expected Result → "status PENDING"
           └─ Reason: App behavior changed, TC must reflect reality

        B) TC is correct: App has defect
           └─ Create: defect ticket
           └─ Description: "Create employee returns PENDING, should be DRAFT"
           └─ Reference: TC DF-M1-POS-005
           └─ Action: Fix app, then test passes

        C) Confusion on requirement:
           └─ Escalate to Product/QA
           └─ Clarify: What should status be after create?
           └─ Update: TC based on clarification

NEVER (repeat: NEVER):
  ❌ "Let me just change assertion to PENDING"
  ❌ "The test passes now"
```

> ❌ This is AUTOMATION FORGERY
> ❌ Root cause: App behavior ≠ TC requirement
> ❌ Hiding it = covering up defect

Decision Tree: What to Do When Test Fails?

```
TEST FAILED
    ↓
╔═══════════════════════════════════════════════════════════════╗
║ STEP 1: Identify which LAYER failed                           ║
╠═══════════════════════════════════════════════════════════════╣
║                                                               ║
║  A) PRE (Precondition)                                        ║
║     └─ Setup failed, test scenario cannot run                 ║
║     └─ Check: /v1/admin/seed/user endpoint                    ║
║     └─ Check: Database state after PRE                        ║
║     └─ Action: Fix environment, then RETRY                    ║
║     └─ TC Status: remains PLANNED (not automated)             ║
║                                                               ║
║  B) FACTORY (Data Generation)                                 ║
║     └─ Generated data invalid                                 ║
║     └─ Check: Data matches schema?                            ║
║     └─ Check: Data unique (no conflicts)?                     ║
║     └─ Action: RETRY (usually one-off)                        ║
║                └─ If repeats: enhance FACTORY validation      ║
║                                                               ║
║  C) USECASE (Business Logic)                                  ║
║     └─ Logic returned error                                   ║
║     └─ Check: Error expected per TC?                          ║
║     └─ Check: USECASE logic matches TC?                       ║
║     └─ Check: Input data valid?                               ║
║     └─ Action: FIX USECASE or DB cleanup                      ║
║                                                               ║
║  D) Layer 3 (PAGE/COMPONENT/Endpoint)                         ║
║     └─ Selector not found OR endpoint 404/500                 ║
║     └─ Check: Selector exists in current app?                 ║
║     └─ Check: Endpoint still exists?                          ║
║     └─ Check: Grounding outdated?                             ║
║     └─ Action: RE-GROUND selector/endpoint                    ║
║                └─ Or: UPDATE Layer 3 code                     ║
║                                                               ║
║  E) SPEC (Assertion)                                          ║
║     └─ Expected value ≠ Actual value                          ║
║     └─ Check: TC expected result accurate?                    ║
║     └─ Check: App behavior matches TC?                        ║
║     └─ Decision: UPDATE TC or REPORT DEFECT                   ║
║     └─ 🚫 NEVER adjust assertion to pass                      ║
║                                                               ║
╚═══════════════════════════════════════════════════════════════╝
```

```
    ↓
╔═══════════════════════════════════════════════════════════════╗
║ STEP 2: Gather evidence                                       ║
╠═══════════════════════════════════════════════════════════════╣
║                                                               ║
║  Screenshot: outputs/evidence/[YYYY-MM-DD]/[TC-ID].png        ║
║  Video: outputs/evidence/[YYYY-MM-DD]/[TC-ID].webm            ║
║  Console logs: Browser errors                                 ║
║  Network logs: HTTP requests/responses                        ║
║  Stack trace: Full error stack                                ║
║  Command: Exact command that was run                          ║
║                                                               ║
╚═══════════════════════════════════════════════════════════════╝
    ↓
╔═══════════════════════════════════════════════════════════════╗
║ STEP 3: Root cause analysis                                   ║
╠═══════════════════════════════════════════════════════════════╣
║                                                               ║
║  ❓ What exactly failed?                                      ║
║     └─ Not "test failed", but "Layer X → reason Y"            ║
║                                                               ║
║  ❓ Why did it fail?                                          ║
║     └─ Selector wrong? Endpoint not responding? Data invalid? ║
║                                                               ║
║  ❓ Is failure expected per TC?                               ║
║     └─ If yes: Assertion should verify it                     ║
║     └─ If no: Something is broken                             ║
║                                                               ║
╚═══════════════════════════════════════════════════════════════╝
    ↓
╔═══════════════════════════════════════════════════════════════╗
║ STEP 4: Decision & Action                                     ║
╠═══════════════════════════════════════════════════════════════╣
║                                                               ║
║  BLOCKER (do not pass):                                       ║
║    ❌ PRE setup failed – environment issue                    ║
║    ❌ SPEC assertion fail (app ≠ TC) – needs decision         ║
║    ❌ Layer 3 selector wrong – needs fix                      ║
║    ❌ @covers TC-ID missing – spec issue                      ║
║                                                               ║
║  ACTIONABLE (fix and retry):                                  ║
║    ✅ FACTORY data invalid – enhance FACTORY                  ║
║    ✅ USECASE error (expected) – fix if TC changed            ║
║    ✅ Layer 3 outdated – re-ground and update                 ║
```

```
╔═══════════════════════════════════════════════════════════════╗
║                                                               ║
║  DECISION NEEDED (escalate):                                  ║
║    ⚠️  App behavior ≠ TC expected — product decision          ║
║    ⚠️  Requirement unclear — QA lead decision                 ║
║    ⚠️  Environment down — ops decision                        ║
║                                                               ║
╚═══════════════════════════════════════════════════════════════╝
```

Evidence Requirements

Ketika test gagal, HARUS attach evidence:

Screenshot

```
Lokasi: outputs/evidence/[YYYY-MM-DD]/
Nama: [TC-ID]__[spec-name]__[failure-point].png

Contoh: PM-C-001__create-employee__selector-not-found.png

Requirement:
  ✅ Show DOM state saat gagal
  ✅ Show error message (kalau ada di UI)
  ✅ Include browser dev tools console (errors visible)
  ✅ Timestamp visible (metadata bar)
  ✅ Full viewport (tidak cropped)
```

Video

```
Lokasi: outputs/evidence/[YYYY-MM-DD]/
Nama: [TC-ID]__[spec-name]__[scenario].webm

Contoh: PM-C-001__create-employee__full-flow.webm

Requirement:
  ✅ Full test execution (PRE → scenario → failure)
  ✅ Show exact moment of failure (pause frame)
  ✅ Include network tab (HTTP calls visible)
  ✅ Duration < 5 minutes (reasonable length)
  ✅ Format: webm or mp4
```

Logs

```
Browser Console:
  - All errors printed
  - All network errors
  - All warnings related to test
  - Example:
    Error: fill: element not found <selector>
    at page.fill() [createEmployee.page.ts:42]

Network Log:
  - All HTTP requests
  - Response status + headers
  - Response body (JSON/error)
  - Timing info
  - Example:
    POST /v1/employees HTTP 500
    Response: { error: "Validation failed" }
    Timing: 234ms

Stack Trace:
  - Full stack (bukan truncated)
  - Include file paths + line numbers
  - Include layer info (Layer3 / USECASE / SPEC)
```

Run Information

```
Command:
  playwright test domain/web/pm/tests/create-employee.spec.ts -g "PM-C-001"

Output:
  1 failed (42.3s)
  [chromium] › domain/web/pm/tests/create-employee.spec.ts:42:3
  › 01  [UI] Create employee with valid data

  Error: Timeout 30000ms exceeded.
```

Reporting to Team/Channel

❌ BAD Report:

```
"Test PM-001 gagal"
"Ada error di create employee"
"Selector tidak ketemu, kelihatannya app berubah"
"Udah coba diulangi tapi tetap gagal"
```

✅ GOOD Report:

```
TEST FAILURE: DF-M1-POS-001

@covers: DF-M1-POS-001 — Create employee with valid data
Layer: Layer 3 — PAGE.fillName()
Error: Timeout 30000ms | selector [data-testid="employee-name"] not found

ROOT CAUSE:
  Grounding found: [data-testid="employee-name"]
  Live app shows: [data-testid="first-name"]
  Diagnosis: Selector outdated OR app changed

EVIDENCE:
  Screenshot: outputs/evidence/2025-01-15/DF-M1-POS-001.png
  Video: outputs/evidence/2025-01-15/DF-M1-POS-001.webm
  Command: playwright test domain/web/pm/tests/create-employee.spec.ts -g "DF-M1

ACTION NEEDED:
  Option A: Fix PAGE layer → update selector to [data-testid="first-name"]
  Option B: Re-ground → verify selector against current app
  Option C: Check TC → does TC describe "employee name" or "first name"?

ASSIGNMENT:
  Automation engineer: Update selector if app UI finalized
  QA lead: Verify TC accuracy
  Developer: Confirm UI is intentional change
```

Checklist: Sebelum Report "Test Failed"

```
REQUIRED:
  [ ] @covers TC-ID ada dan benar?
  [ ] Tahu layer mana yang fail? (PRE/FACTORY/USECASE/Layer3/SPEC)
  [ ] Error message lengkap (bukan truncated)?
  [ ] Evidence attached (screenshot/video)?
  [ ] Root cause analyzed (bukan "dunno")?
  [ ] Decision path clear (action item defined)?
  [ ] Command yang dijalankan dicatat?

RED FLAGS (do not pass):
  [ ] "Assertion agar pass" — NEVER
  [ ] "Silent failure" — NEVER
  [ ] "Exception ditelan" — NEVER
  [ ] "Tautology assertion" — expect(true).toBe(true)
  [ ] "@covers missing" — BLOCKER
  [ ] "Tidak ada evidence" — BLOCKER
  [ ] "Root cause tidak jelas" — BLOCKER
```

Anti-Patterns: What NOT to Do

❌ Silent Failures

```
WRONG:
  test('Create employee', async () => {
    try {
      await page.fill('[data-testid="name"]', 'John')
    } catch (e) {
      console.log(e)  // Error logged but test continues
    }
    expect(true).toBe(true)  // Always passes
  })

Result: Test passes even though selector failed

RIGHT:
  test('Create employee', async () => {
    await page.fill('[data-testid="name"]', 'John')
    // No try/catch - let error propagate
    expect(page.locator('[data-testid="name"]')).toHaveValue('John')
    // Explicit assertion
  })
```

❌ Tautology Assertions

```
WRONG:
  expect(result).toBeDefined()  // Always true if reached
  expect(result.id > 0).toBeTruthy()  // True for any positive number
  expect(true).toBe(true)  // Always passes

RIGHT:
  expect(result.status).toBe('DRAFT')  // Can fail if status is PENDING
  expect(result.id).toBe(expectedId)  // Specific value check
  expect(employees).toHaveLength(expectedCount)  // Countable
```

❌ Assertion Forgery

```
WRONG:
  // TC says: status DRAFT
  // App returns: status PENDING
  // Solution: change assertion

  expect(result.status).toBe('PENDING')  ← FORGERY
  // Now test passes, but TC requirement is violated

RIGHT:
  // Keep assertion as per TC
  expect(result.status).toBe('DRAFT')

  // If test fails: decision point
  - Is TC wrong? Update TC (with approval)
  - Is app wrong? Report defect (fix app)
  - Never silently change assertion
```

❌ Hidden Dependencies

```
WRONG:
  test('Create', async () => {
    const result = await create(data)  // Assume success
    expect(result.id).toBeTruthy()  // But what if create failed?
  })

Result: If create returns error, assertion misleading

RIGHT:
  test('Create', async () => {
    const result = await create(data)
    expect(result.success).toBe(true)  // Verify create succeeded
    expect(result.id).toBeTruthy()  // Then verify ID exists
  })
```

Summary: Executability Principle + Debuggability

```
Automation testing prinsip:

1. EXECUTABLE
   └─ Test harus bisa run di dev/stg
   └─ Tidak ada assumption, semua verified

2. DEBUGGABLE
   └─ Kalau fail, error jelas dan actionable
   └─ Harus tahu: layer mana? kenapa? apa action-nya?

3. TRACEABLE
   └─ Error → @covers TC → root cause
   └─ Evidence lengkap (screenshot/video)

4. ACCOUNTABLE
   └─ Tidak ada silent failures
   └─ Tidak ada assertion forgery
   └─ Tidak ada "magic" atau assumption
```

Next Document

Dokumentasi ini mengasumsikan test sudah written dan dijalankan. Untuk error handling di **grounding phase** dan **layer design phase**, lihat: - `docs/shared/03-codebase-grounding.md` — ground dulu sebelum code - `docs/shared/04-three-layer-architecture.md` — DDD rules per layer - `workflow/01-grounding.md` — grounding phase detail

---

16 — Assertion design & error handling

Goal: Design assertion and error handling from TC BEFORE writing code. Prevent tautology and swallowed errors upstream.

Rule 1: Every Assertion Must Be Able to FAIL

```
❌ TAUTOLOGY (always pass):
  expect(true).toBe(true)
  expect(result).toBeDefined()
  expect(count).toBeGreaterThanOrEqual(0)

✅ MEANINGFUL (can fail):
  expect(result.status).toBe('DRAFT')
  expect(employees).toHaveLength(3)
  expect(form).toBeVisible()
```

Rule 2: Every Error Must PROPAGATE

```
❌ SWALLOWED (error hidden):
  .catch(() => false)
  .catch(() => {})
  try { } catch(e) { console.log(e) }
  result.success || true

✅ PROPAGATE (error clear):
  // Let error throw naturally
  throw new Error(`Failed: ${message}`)
  if (!response.ok()) throw new Error(...)
```

Rule 3: Every Value from TC (not code)

```
❌ HARDCODED (from memory):
  expect(result.status).toBe('ACTIVE')  // code assumes
  const name = 'John'  // hardcoded

✅ FROM TC (traceable):
  expect(result.status).toBe('DRAFT')  // from TC Then#1
  const data = factory.generateEmployee()  // varied, from FACTORY
```

Assertion Design Template (Grounding Phase)

For each TC Then/And clause:

TC Clause	Observable	Expected	Assertion	Can Fail?
"Status DRAFT"	result.status field	"DRAFT"	expect(...).toBe('DRAFT')	✅ Yes
"Row exists"	row in table	row.id === X	expect(rows).toContainEqual({id: X})	✅ Yes
"Form visible"	form element	visible	expect(form).toBeVisible()	✅ Yes

---

Error Handling Pattern (per Layer)

SPEC Layer

```
test('@covers DF-M1-POS-001', async () => {
  // Let error propagate (not swallow)
  const result = await usecase.execute(data)

  // Assertion: can fail if feature broken
  expect(result.status).toBe('DRAFT')
})
```

USECASE Layer

```
export class CreateEmployeeUsecase {
  async execute(data: EmployeeInput): Promise<UsecaseResult> {
    try {
      const res = await http.post('/v1/employees', data)
      if (!res.ok()) {
        throw new Error(`API error: ${res.status()}`)
      }
      return { success: true, data: res.json() }
    } catch (e) {
      // Re-throw with context (not swallow)
      throw new Error(`Create failed: ${e.message}`)
    }
  }
}
```

Layer 3 (PAGE)

```
async fillName(name: string) {
  // Let error propagate (not .catch)
  await this.page.fill('[data-testid="name"]', name)

  // Positive anchor (not negative read)
  await expect(this.page.locator('[data-testid="name"]')).toHaveValue(name)
}
```

FACTORY

```
export function generateEmployee(overrides = {}) {
  // Validate data (not assume valid)
  const data = {
    name: faker.person.fullName(),
    email: faker.internet.email(),
    ...overrides
  }

  // Optional: validate schema
  if (!data.email.includes('@')) {
    throw new Error('Invalid email generated')
  }

  return data
}
```

PRE (Precondition)

```
export async function preconditionManagerUser() {
  const response = await request.post('/v1/admin/seed/user', {
    role: 'manager',
    email: `mgr-${Date.now()}@test.co`
  })

  // Assert PRE executed (not assume)
  expect(response.status()).toBe(201)
  expect(response.json().id).toBeTruthy()

  return response.json()
}
```

Checklist: Before Implementing

```
Assertion Design:
  [ ] No tautology (not expect(true).toBe(true))
  [ ] Can fail (jika feature broken)
  [ ] From TC expected, not code
  [ ] // expect: <TC-ID> Then#n comment added
  [ ] One assertion per TC clause

Error Handling:
  [ ] No .catch(() => false)
  [ ] No try/catch hiding error
  [ ] Errors propagate with message
  [ ] PRE assertion explicit
  [ ] USECASE error handling clear

Data & Values:
  [ ] No hardcoded data (use FACTORY)
  [ ] Dynamic email/id per run
  [ ] Expected value from TC, not memory
  [ ] Selector from grounding (// src:)

DDD:
  [ ] SPEC calls usecase only
  [ ] USECASE no expect()
  [ ] Layer 3 no business logic
  [ ] FACTORY no API calls
```

Red Flags During Code Review

```
BLOCKER:
  ❌ Tautology: expect(true), toBeDefined(), toBeGreaterThanOrEqual(0)
  ❌ Swallowed error: .catch(() => false)
  ❌ Hardcoded value: 'John Doe', 'john@test.co'
  ❌ Missing assertion: TC clause with no expect()
  ❌ DDD broken: SPEC touch page, USECASE with expect()
  ❌ No error handling: try/catch with console.log
```

Reference

- Executability: `docs/shared/14-executability-principle.md`
- Test design: `docs/shared/05-test-design-rules.md`
- Layer architecture: `docs/shared/04-three-layer-architecture.md`
- Failure reporting: `docs/shared/15-test-failure-reporting.md`

---

17 — Traceability record: Automation to TC

Goal: Every automation must be traceable back to canonical TC. Record verification in centralized checklist.

What to Record

1. AUTOMATION-CHECKLIST-.md (Centralized)

Location: `domain/[web|mobile|api]/[module]/AUTOMATION-CHECKLIST-<TC-ID>.md`

Example: `domain/web/pm/AUTOMATION-CHECKLIST-DF-M1-POS-001.md`

Template: AUTOMATION-CHECKLIST

```
# Automation Checklist: <TC-ID>

TC: <TC-ID> – <TC Title>
Automation engineer: [name]
Date: [YYYY-MM-DD]

## Given (Preconditions)

- [ ] PRE: [what precondition]
  - Factory: `preconditionXxx()`
  - Location: [file:line]
  - Endpoint: [POST /v1/...]
  - Status code: 201 ✓
  - DB mutation verified: ✓

## When (Actions)

- [ ] Action: [navigate/fill/click/submit]
  - Code: [file:line]
  - Selector: [data-testid="..."]
  - Grounded: // src: [codebase/file:line] ✓
  - Can fail: YES ✓

## Then/And (Expected Results)

### Then#1: [TC clause]
- [ ] Assertion: `expect(...).toBe(...)`
  - Location: [file:line]
  - Comment: // expect: <TC-ID> Then#1
  - Can fail: YES ✓
  - From TC: YES ✓
  - Tautology: NO ✓

### And#1: [TC clause]
- [ ] Assertion: `expect(...).toBe(...)`
  - Location: [file:line]
  - Comment: // expect: <TC-ID> And#1
  - Can fail: YES ✓
  - From TC: YES ✓

## Red Flag Prevention

- [ ] Tautology: NO ✓
- [ ] Swallowed error: NO ✓
```

```
- [ ] Hardcoded value: NO ✓
- [ ] DDD broken: NO ✓

## Summary

- Total TC clauses: [N]
- Assertions implemented: [N/N] ✓
- Coverage: 100% ✓
- Status: READY FOR PR ✓
```

---

2. Code Comments (Distributed)

Scattered dalam SPEC/USECASE/PAGE/FACTORY:

```
// @covers <TC-ID>
test('...', async () => {
  // // expect: <TC-ID> Then#1
  expect(...).toBe(...)
})

// src: filepath:line (selector)
await page.fill('[data-testid="..."]', value)
```

---

3. Pre-push Checklist

In `docs/shared/11-pre-push-review.md`:

```
[ ] AUTOMATION-CHECKLIST-<TC-ID>.md exist?
[ ] All TC clauses covered?
[ ] Code comments (@covers, // expect:, // src:) present?
[ ] Red flags scanned (tautology, swallowed error)?
[ ] Test result: PASSED?
```

Verification Process

Phase 4: Implement

> 1. Write code (SPEC/USECASE/Layer3/FACTORY)
> 2. Add comments (@covers, // expect:, // src:)
> 3. Create AUTOMATION-CHECKLIST-<TC-ID>.md
> 4. Fill checklist: every TC clause → assertion mapping
> 5. Self-verify: can fail? meaningful? from TC?
> 6. Self-sign: engineer mark ✓

Phase 5: Verify

> 1. Run test → PASSED (or debug + fix)
> 2. Verify PRE executed ✓
> 3. Collect evidence (video, logs)
> 4. Check AUTOMATION-CHECKLIST complete ✓

Phase 6: PR Review

> 1. Read AUTOMATION-CHECKLIST
> 2. Confirm: verification done upstream ✓
> 3. No blocker: APPROVED
> 4. Merge → Automation status = true

Red Flags in Checklist

If any of these unchecked = REQUEST_CHANGES:

> ❌ Total TC clauses ≠ Assertions implemented
> → Missing assertion for some clause
>
> ❌ Tautology: YES
> → Fix: make assertion meaningful
>
> ❌ Swallowed error: YES
> → Fix: let error propagate
>
> ❌ Hardcoded value: YES
> → Fix: use FACTORY or from TC
>
> ❌ Can fail: NO
> → Fix: assertion must be able to fail
>
> ❌ From TC: NO
> → Fix: expected value must be from TC
>
> ❌ Test result: FAILED or SKIPPED
> → Fix: run test, get PASSED only

Sample AUTOMATION-CHECKLIST

```
# Automation Checklist: DF-M1-POS-001

TC: DF-M1-POS-001 — Create employee with valid data
Automation engineer: [name]
Date: 2026-01-15

## Given (Preconditions)

- [x] PRE: Manager user exists
  - Factory: `preconditionManagerUser()`
  - Location: domain/web/pm/factories/preconditions.factory.ts:23
  - Endpoint: POST /v1/admin/seed/user
  - Status code: 201 ✓
  - DB mutation verified: users.role = 'manager' ✓

## When (Actions)

- [x] Navigate to form
  - Code: domain/web/pm/tests/create-employee.spec.ts:15
  - Selector: [data-testid="create-form"]
  - Grounded: // src: kerjaan-client-portal/src/pages/.../CreateEmployee.tsx:42

- [x] Fill name
  - Code: domain/web/pm/tests/create-employee.spec.ts:18
  - Selector: [data-testid="employee-name"]
  - Data: 'John Smith' (from TC Given) ✓

- [x] Submit
  - Code: domain/web/pm/tests/create-employee.spec.ts:21
  - Wait: networkidle ✓

## Then/And (Expected Results)

### Then#1: "Status DRAFT"
- [x] Assertion: `expect(result.status).toBe('DRAFT')`
  - Location: domain/web/pm/tests/create-employee.spec.ts:26
  - Comment: // expect: DF-M1-POS-001 Then#1 ✓
  - Can fail: YES ✓
  - From TC: YES ✓
  - Tautology: NO ✓

### Then#2: "Employee ID returned"
- [x] Assertion: `expect(result.id).toBeTruthy()`
  - Location: domain/web/pm/tests/create-employee.spec.ts:27
```

```
  - Comment: // expect: DF-M1-POS-001 Then#2 ✓
  - Can fail: YES ✓

## Red Flag Prevention

- [x] Tautology: NO ✓
- [x] Swallowed error: NO ✓
- [x] Hardcoded value: NO ✓
- [x] DDD broken: NO ✓

## Summary

- Total TC clauses: 2 (2 Then)
- Assertions implemented: 2/2 ✓
- Coverage: 100% ✓
- Status: READY FOR PR ✓

Self-reviewed by: [engineer name]
Date: 2026-01-15 ✓
```

Reference

- Assertion design: `docs/shared/16-assertion-design.md`
- Pre-push review: `docs/shared/11-pre-push-review.md`
- Implementation: `workflow/04-implement.md`
- Failure reporting: `docs/shared/15-test-failure-reporting.md`

---

22 — Automation Fixing: Best Practices for Runtime Failures

Goal: When automation fails at runtime, systematically diagnose and fix using canonical TC as source of truth.

🚨 Critical Principle

> Automation failures have 3 possible causes:
>
> 1. ❌ TEST CODE/SETUP BUG
>   → Fix automation code + setup/data
>   → Keep TC unchanged (TC is correct)
>
> 2. ❌ TC DIFFERS FROM IMPLEMENTATION
>   → Document difference clearly
>   → Decide: update TC or product defect?
>   → Don't silently accept wrong result
>
> 3. ⚠️ PRODUCT DEFECT
>   → Keep TC (TC requirement correct)
>   → Report product bug with evidence
>   → Block automation until fixed
>   → Don't make test "pass" by accepting defect
>
> NEVER: Make test pass by changing expected value to match wrong result ❌

---

10 Best Practices for Automation Fixing

1️⃣ Run Actual Scenarios, Not Just Checks

❌ WRONG:

```
npm run lint
npm run type-check
npm run test -- --collect-only  # Just collect, no execute
```

✅ RIGHT:

```
npm test -- create-employee.spec.ts --project=chromium
# Actually run, watch it execute, see what happens
```

Why: Type-check passes ≠ automation works. Only real execution reveals runtime issues.

2️⃣ Run Sequentially, No Retry, Stop at First Failure

❌ WRONG:

```
npm test -- create-employee.spec.ts --retries=3  # Retry failures
npm test -- --parallel=4  # Run in parallel, hide failures
```

✅ RIGHT:

```
npm test -- create-employee.spec.ts  # Single run, sequential
# First failure: STOP, diagnose, fix
# Then rerun
```

Why: - Retries hide flaky tests (band-aid, not fix) - Parallel runs hide order/state dependencies - First failure is where the real problem is

---

3️⃣ When Test Fails: Open TC and Document

Step 1: Open canonical TC from staffinc-testcase

```
# Test Case: DF-M1-POS-001

TC Location: staffinc-testcase/testcase/web/pm/DF-M1-POS-001.md

**Given (Preconditions):**
- Manager user exists
- Form endpoint available

**When (Actions):**
1. Navigate to /pm/employees/create
2. Fill name = 'John Smith'
3. Fill email = 'john@test.co'
4. Click submit

**Then (Expected):**
- Record created in DB
- Status = DRAFT (from requirement)
- Employee ID returned
```

Step 2: Document failure context

````
# Failure Report: DF-M1-POS-001

## Runtime Failure

**Test:** @covers DF-M1-POS-001
**Failure:** Expected status = 'DRAFT', got 'PENDING'
**Error:** AssertionError: expect(status).toBe('DRAFT')

## Reproduction

### Preconditions (from TC Given):
- [✓] Manager user: email=mgr-123@test.co, id=user_456
- [✓] Form endpoint: GET /v1/pm/create-form → 200 OK
- [✓] DB state: clean, no existing records

### Reproduction Steps (from TC When):
1. Navigate: http://localhost:3000/pm/employees/create
   Result: Form loaded ✓
2. Fill name: 'John Smith'
   Observed: Input shows 'John Smith' ✓
3. Fill email: 'john@test.co'
   Observed: Input shows 'john@test.co' ✓
4. Click submit button
   Observed: Form submitted, HTTP 201 received ✓

### Expected (from TC Then):
```json
{
  "id": "emp_xxx",
  "status": "DRAFT",
  "name": "John Smith",
  "createdBy": "mgr-123"
}
````
````

Actual (from runtime):

```
{
  "id": "emp_789",
  "status": "PENDING",    // ❌ WRONG: should be DRAFT
  "name": "John Smith",
  "createdBy": "mgr-123"
}
```

Error Details:

```
Error: expect(status).toBe('DRAFT')
Expected: 'DRAFT'
Actual: 'PENDING'
Location: create-employee.spec.ts:26
```

Evidence:

- Screenshot: outputs/evidence/failure-screenshot.png
- Video: outputs/evidence/failure-video.webm
- Logs: outputs/logs/test-failure.log

```
---

### **4️⃣ Trace Codebase to Find Explanation**

**Compare TC requirement with actual implementation:**
```

TC Expected: status = 'DRAFT' Actual: status = 'PENDING'

Where does status get set? → Search codebase for "PENDING"

```
**Search & find:**
```

$ grep -r "PENDING" domain/web/pm/ domain/web/pm/usecases/ create-employee.usecase.ts:42 if (request.source === 'import') { employee.status = 'PENDING' // ← HERE! } else { employee.status = 'DRAFT' }

> **Analyze:**
>
> ```markdown
> ## Code Trace
>
> ### File: domain/web/pm/usecases/create-employee.usecase.ts:42
>
> ```typescript
> export class CreateEmployeeUsecase {
>   async execute(data: EmployeeInput) {
>     const employee = new Employee(data)
>
>     // Line 42: Status logic
>     if (request.source === 'import') {
>       employee.status = 'PENDING'  // ← Bug: where does 'request' come from?
>     } else {
>       employee.status = 'DRAFT'
>     }
>
>     await db.employees.insert(employee)
>     return employee
>   }
> }

Issue Found:

Line 42 checks `request.source === 'import'` but: - TC Given: User = manager (manual creation via UI) - Code: request.source is NOT passed from UI layer - Result: request.source is undefined - Fallback: `undefined === 'import'` is FALSE - Expected path taken: else → status = 'DRAFT' - BUT: Actual status = 'PENDING'
Hmm, logic says should be DRAFT, but actual is PENDING. Let me check PAGE layer.

> **Continue tracing:**

$ grep -r "'PENDING'" domain/web/pm/ domain/web/pm/pages/create-employee.page.ts:85 domain/web/pm/usecases/create-employee.usecase.ts:42 domain/web/pm/factories/employee-data.factory.ts:12 // ← Suspicious!

> **Found it:**
>
> ```typescript
> // domain/web/pm/factories/employee-data.factory.ts:12
> export function generateEmployee(overrides = {}) {
>   return {
>     name: faker.person.fullName(),
>     email: faker.internet.email(),
>     status: 'PENDING',  // ← HARDCODED! Overrides usecase logic!
>     ...overrides
>   }
> }
> ```

Root Cause: FACTORY hardcodes status = 'PENDING', overriding usecase logic.

5️⃣ Distinguish the Cause

Is it automation/setup/data issue?

> ✓ YES: Factory hardcodes wrong status
> → Fix: Remove hardcoded status from factory
> → FACTORY should NOT set status (let USECASE decide)
> → Scope: Automation code fix (within approval)

OR is TC different from implementation?

> ✗ NO: This is not a TC vs implementation mismatch
> - TC says: status should be DRAFT
> - Implementation says: status should be DRAFT
> - Factory says: status = PENDING (BUG in automation code)

OR is it a product defect?

> ✗ NO: Product code is correct (USECASE sets DRAFT)
> - Automation code (FACTORY) is wrong

6️⃣ After Fixing: Rerun Failed Scenario FIRST

Fix applied:

```
// domain/web/pm/factories/employee-data.factory.ts:12
export function generateEmployee(overrides = {}) {
  return {
    name: faker.person.fullName(),
    email: faker.internet.email(),
    // REMOVED: status: 'PENDING'  ← Now usecase decides
    ...overrides
  }
}
```

Rerun ONLY the failed scenario:

```
npm test -- create-employee.spec.ts --grep "@covers DF-M1-POS-001"
# NOT: Run all tests yet
# WAIT: Verify this ONE scenario passes first
```

Result:

```
✓ Create employee with valid data -> created (45.2s)
✓ Status: DRAFT ✓
✓ ID: emp_123 ✓

→ This scenario FIXED
```

Only THEN run next scenario:

```
npm test -- create-employee.spec.ts --grep "@covers DF-M1-NEG-001"
# Continue with next test
```

7️⃣ Don't Ask User to Debug What Runtime/Codebase Shows

❌ WRONG:

```
Q: "Why is status PENDING instead of DRAFT?
    I don't know, can you check?"
```

✅ RIGHT:

```
Found: FACTORY hardcodes status='PENDING' (file:line)
       but TC expects DRAFT (per usecase logic)
Cause: Line 12 of employee-data.factory.ts
Fix: Removed hardcoded status, let USECASE decide
Rerun: PASSED ✓
```

Only ask user when: - ❓ Requirement decision: "TC says DRAFT, code says PENDING. Which is correct?" - ❓ Access/data unavailable: "Need test account in STG to continue" - ❓ Permission: "Need access to staging environment"

8️⃣ Distinguish Local vs Deployed Code

When reporting: Always mention version

> ## Test Environment
>
> **Local Code:**
> ```bash
> git status
> # On branch feature/create-employee-automation
> # HEAD: 3a1bc2f (fix: remove hardcoded status in factory)

Deployed Code (STG):

> curl https://stg.api.hcm.internal/version
> # {"version": "v2.1.4", "deployedAt": "2026-11-03 10:00 UTC"}

Note: Local code is 1 commit ahead of deployed. - Local: Fix applied ✓ - STG: Still old code (fix not deployed yet)

> **Why:** Local pass ≠ STG pass. Must test both.
>
> ---
>
> ### **9️⃣ Separate Reporting: What's Fixed vs What's Blocked**
>
> ❌ **WRONG:**

"Fixed all issues! Automation ready!" (But actually: only local tests pass, STG not tested, 3 other scenarios not run)

✅ **RIGHT:**
```markdown
## Status Report

### ✓ FIXED
- Scenario DF-M1-POS-001: status issue
  - Root cause: Factory hardcoded status='PENDING'
  - Fix: Removed hardcoding
  - Local rerun: PASSED ✓
  - STG: NOT YET (deploy pending)

### ⏳ BLOCKED
- Scenario DF-M1-POS-002: Not run yet (waiting for DF-M1-POS-001 fix)
- Scenario DF-M1-NEG-001: Not run yet
- STG verification: Waiting for deploy to staging

### 📊 Progress
- Scenarios fixed: 1/3
- Local verified: 1/3
- STG verified: 0/3
- Ready for PR: NO (need STG verification)
```

🔟 Deployment Still Needs Separate Permission

```
Local test pass ✓
  → "Can I deploy?"
  → NO, need deploy permission

Automation fix in code ✓
  → "Can I merge PR?"
  → YES (if review approved)

Code merged to main ✓
  → "Can I deploy?"
  → Separate question, separate permission needed
```

📋 Automation Fixing Checklist

```
Test Failure:
[ ] Run actual scenario (not just type-check)
[ ] Scenario fails? STOP at first failure
[ ] Open canonical TC
[ ] Document: preconditions, steps, expected, actual, error
[ ] Search codebase: trace where issue originates
[ ] Find exact file:line of the problem
[ ] Understand cause: code bug? TC mismatch? product defect?

Fix Applied:
[ ] Fix code (in automation or product?)
[ ] Rerun ONLY the failed scenario
[ ] Scenario now passes? ✓
[ ] Update AUTOMATION-CHECKLIST
[ ] Commit with clear message

Next Scenario:
[ ] Continue with next failed scenario
[ ] OR run full suite if previous fixed
[ ] Report separately: fixed vs blocked

Before PR:
[ ] All scenarios run locally? ✓
[ ] All pass locally? ✓
[ ] STG version tested? ✓
[ ] STG environment confirmed? ✓
[ ] AUTOMATION-CHECKLIST updated? ✓
[ ] Ready for review? ✓
```

🎯 Key Discipline

> NEVER:
> ❌ Retry tests to hide failures
> ❌ Ignore TC and accept wrong result
> ❌ Silently change expected value
> ❌ Deploy local-only fixes
> ❌ Merge without STG verification
>
> ALWAYS:
> ✅ Compare code against TC (source of truth)
> ✅ Run scenarios sequentially, stop at first fail
> ✅ Trace root cause in codebase
> ✅ Report: what's fixed, what's blocked, what's tested
> ✅ Test both locally AND in STG before PR

References

- **20-fix-pr-from-feedback.md** — Fixing after PR review
- **17-traceability-record.md** — AUTOMATION-CHECKLIST tracking
- **14-executability-principle.md** — Understanding layers
- **21-environment-confirmation-stg.md** — STG testing after local